UAH Mars Drone ASW
Architecture Design Document

Index

5 Software Design

5.1 Software Static Architecture

5.2 Software Dynamic Architecture

5.2.1 Computational Model

5.2.2 Software Behaviour

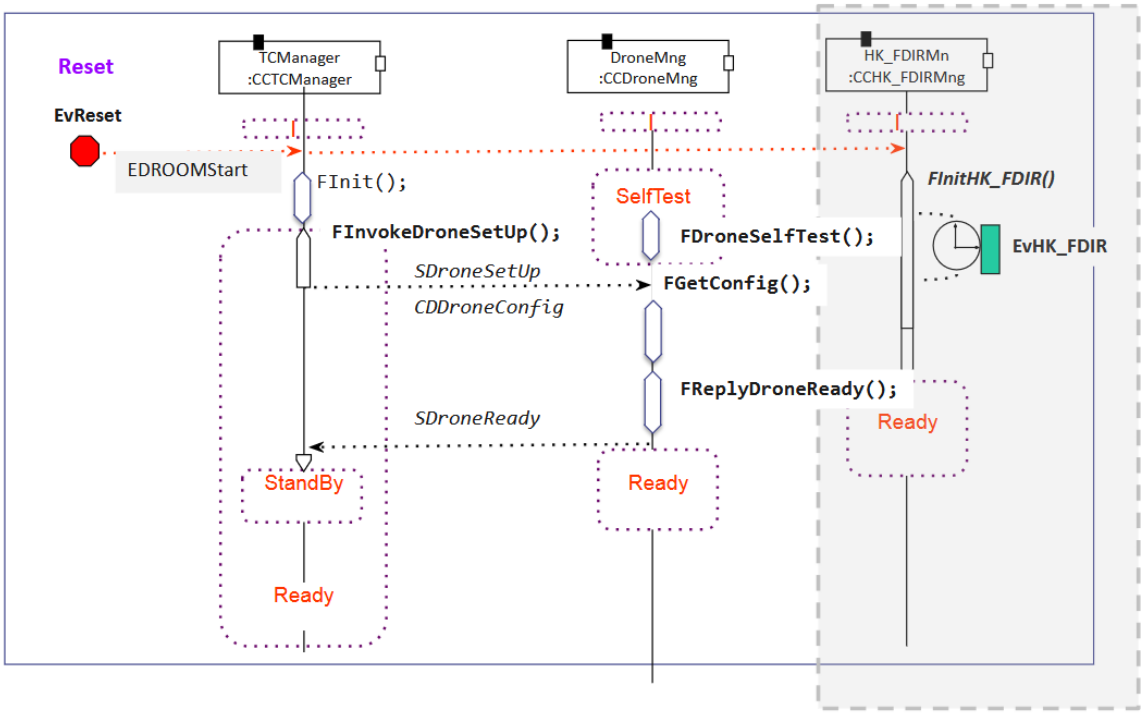
Scenarios

Event	Description
EvRxTC (external IRQ)	TC reception
EvAcceptedTC (internal)	TC acceptation
EvRejectedTC (internal)	TC rejection
EvToReboot (internal)	System Reboot
EvHK_FDIR (timing)	Periodic HK and FDIR Mng
EvHK_FDIR_TC (internal)	HK_FDIR TC Execution
TODO 01: Completa la lista de escenarios y su descripción	
EvReset (reset)	System initialization and drone setup
EvDroneTC (internal)	Drone TC Execution
EvInitFlightPlan (internal)	Initiate the Flight Plan
EvFlightCtrl (timing)	Check periodically if the Flight Plan has ended
EvFlightPlanDone (internal)	Transition to Ready state when Flight Plan is completed

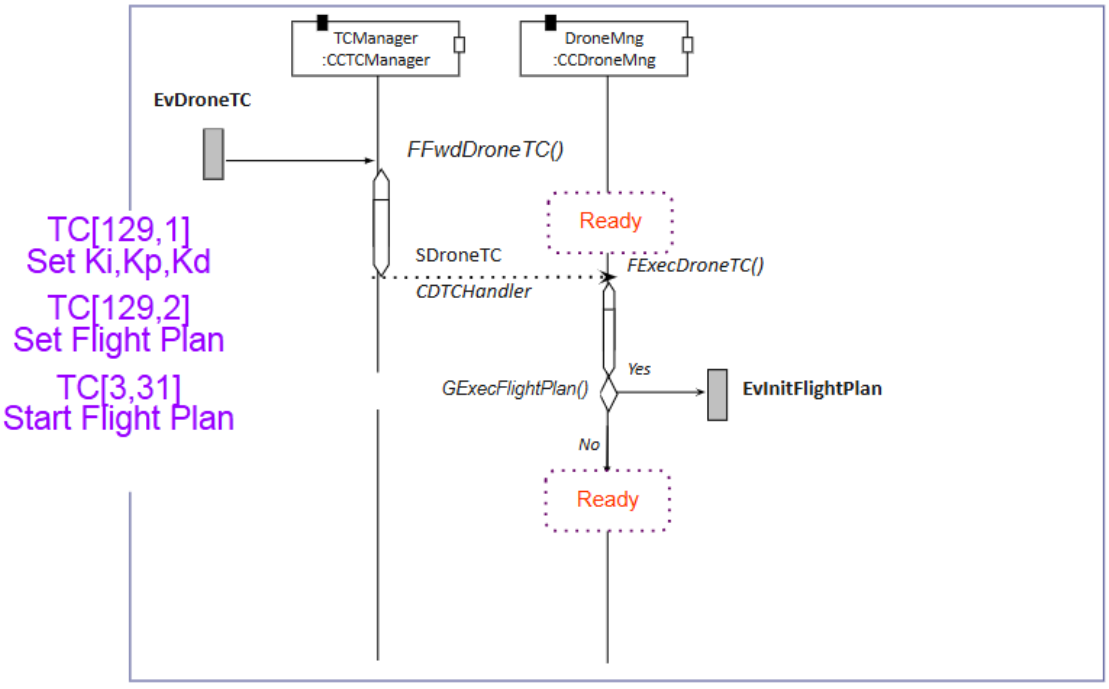
EvDroneTCInFlight (internal)	Drone TC Execution while drone in flight

TODO 02 Añade los diagramas de secuencia de aquellos escenarios ***en los que participa el componente DroneMng***

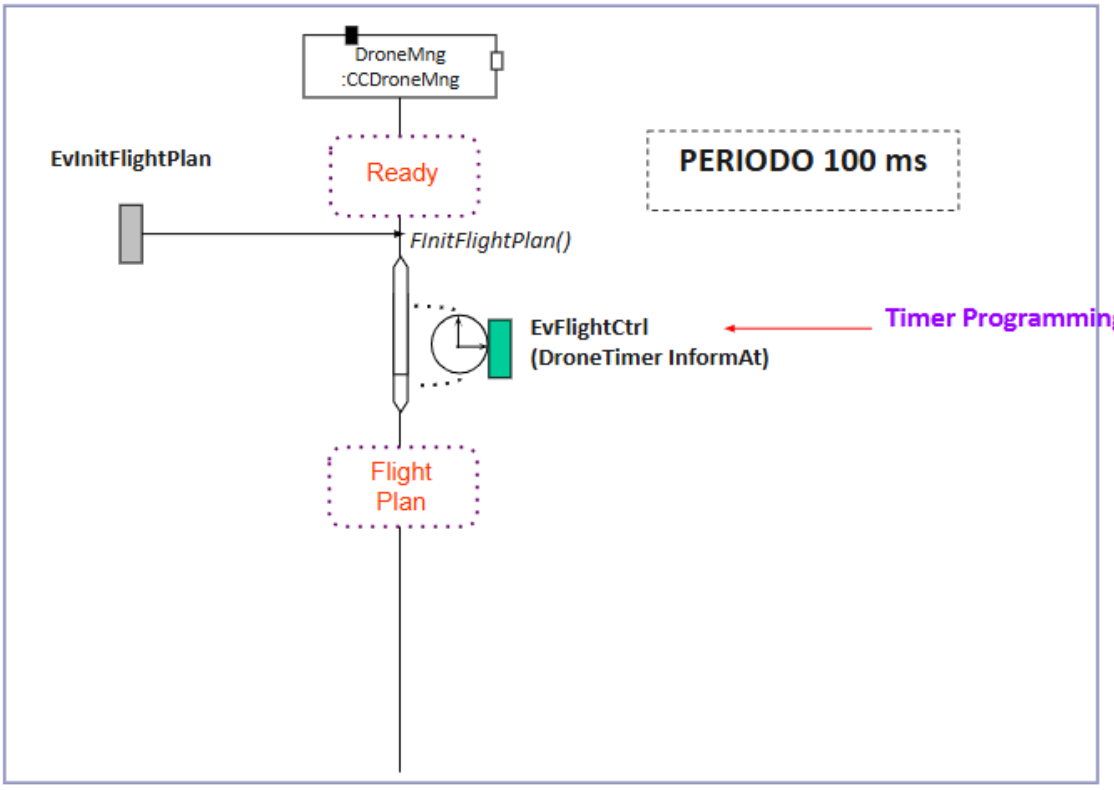
Ev_Reset



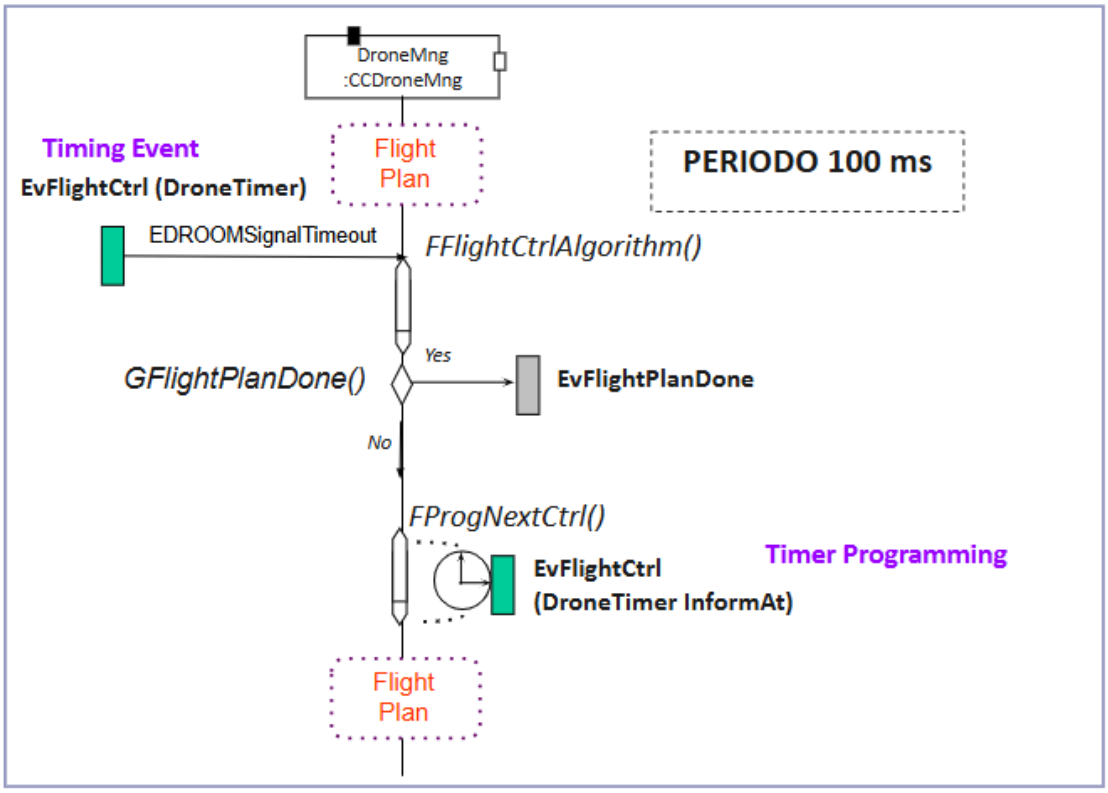
Ev_DroneTC



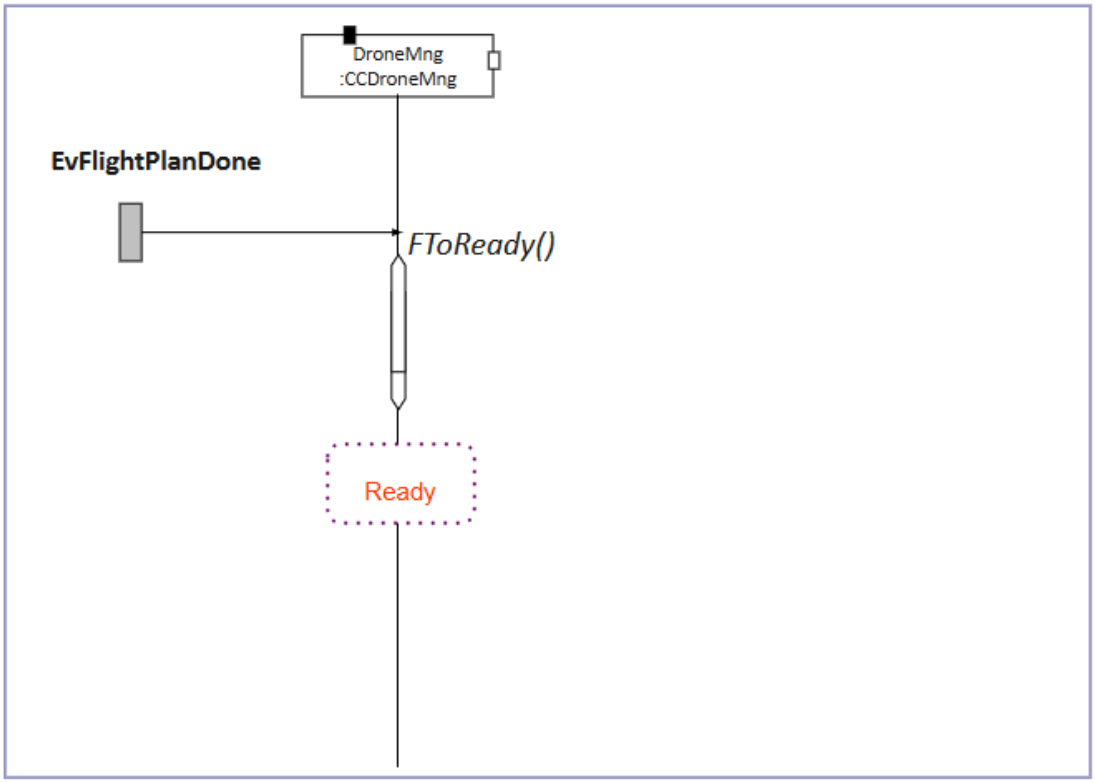
Ev_InitFlightPlan



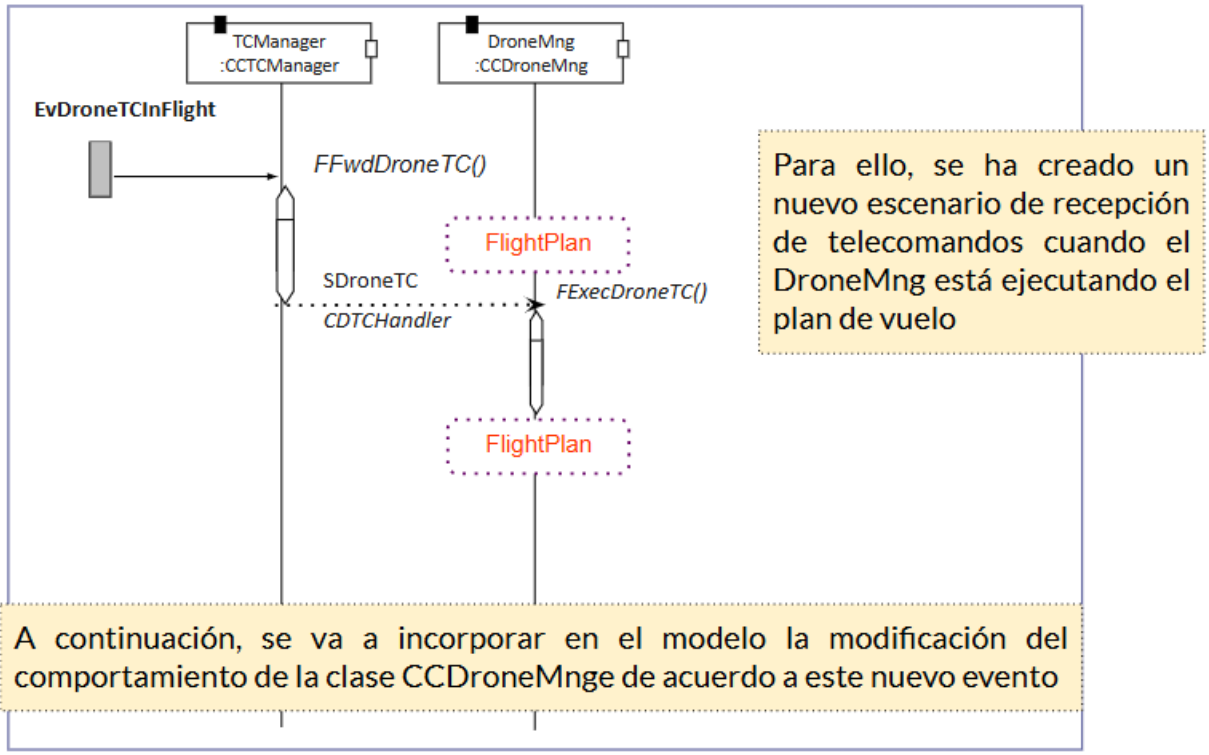
Ev_FlightCtrl



Ev_FlightPlanDone



Ev_DroneTCInFlight



5.3 Real-Time Requirements

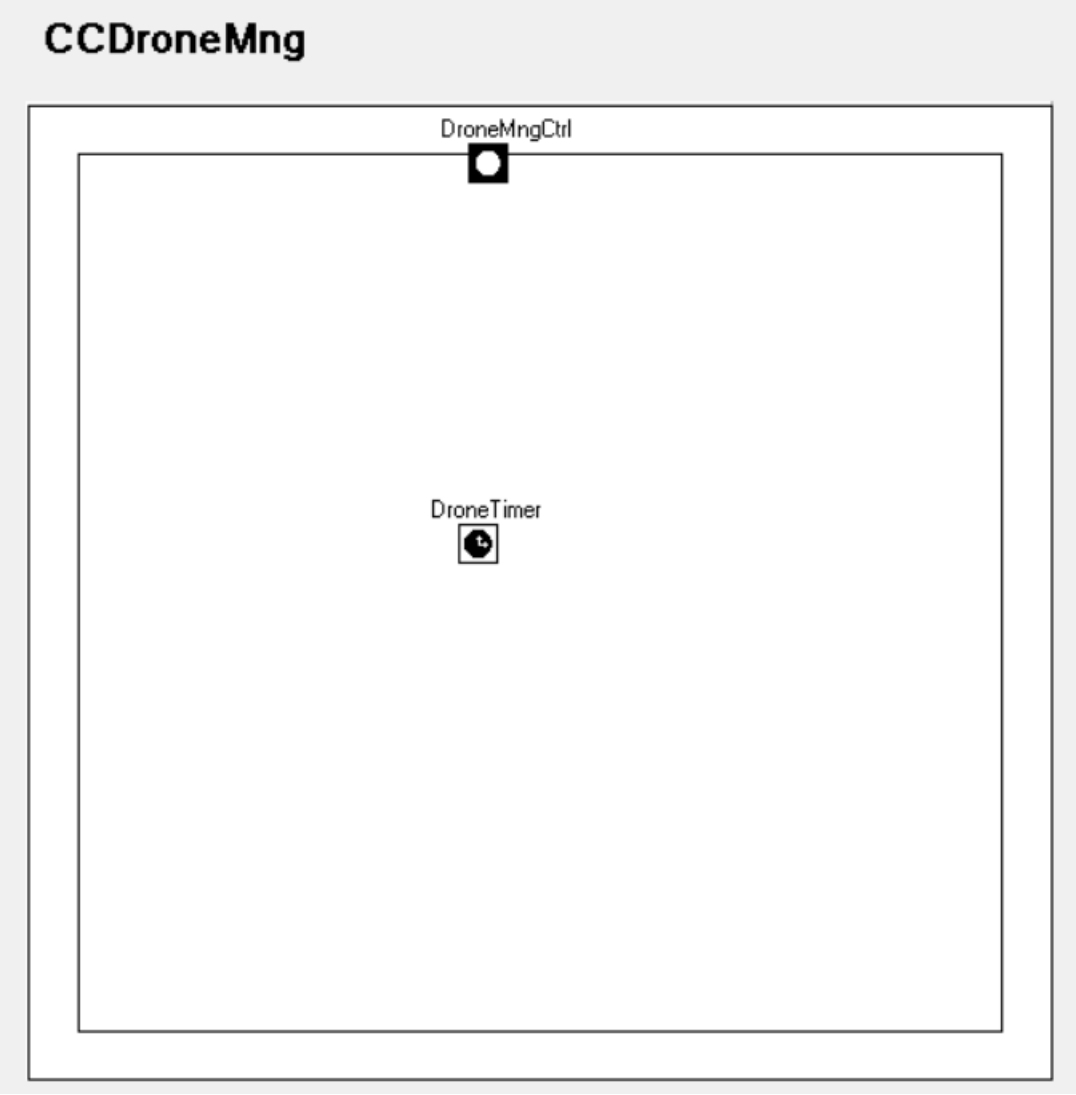
5.4 Software Components Design

5.4.1 CCDroneMng

5.4.1.1 Interfaces

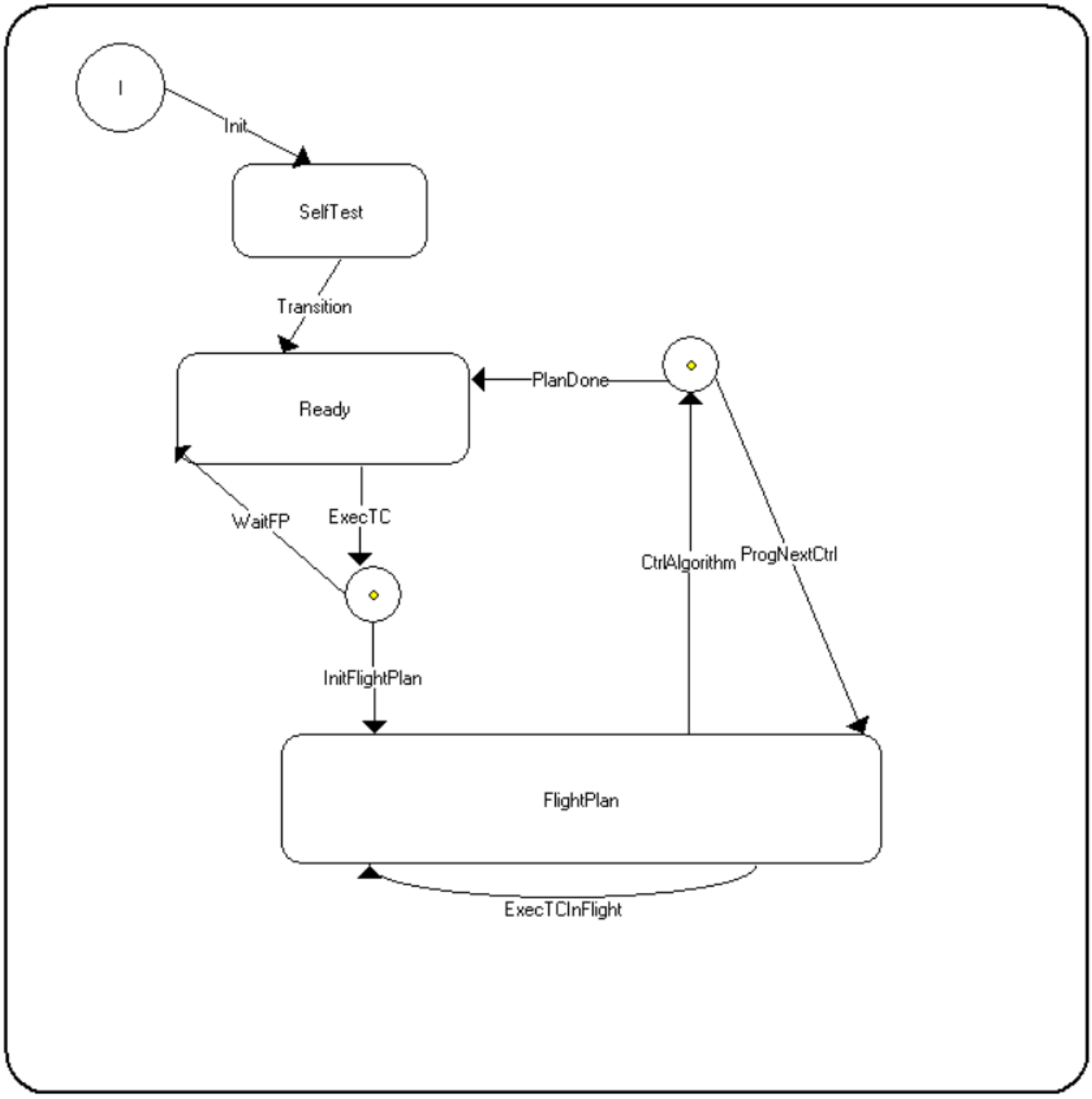
*TODO 03 Añade **los puertos que definen las interfaces de la clase CCDroneMng***

CCDroneMng **Interfaces**

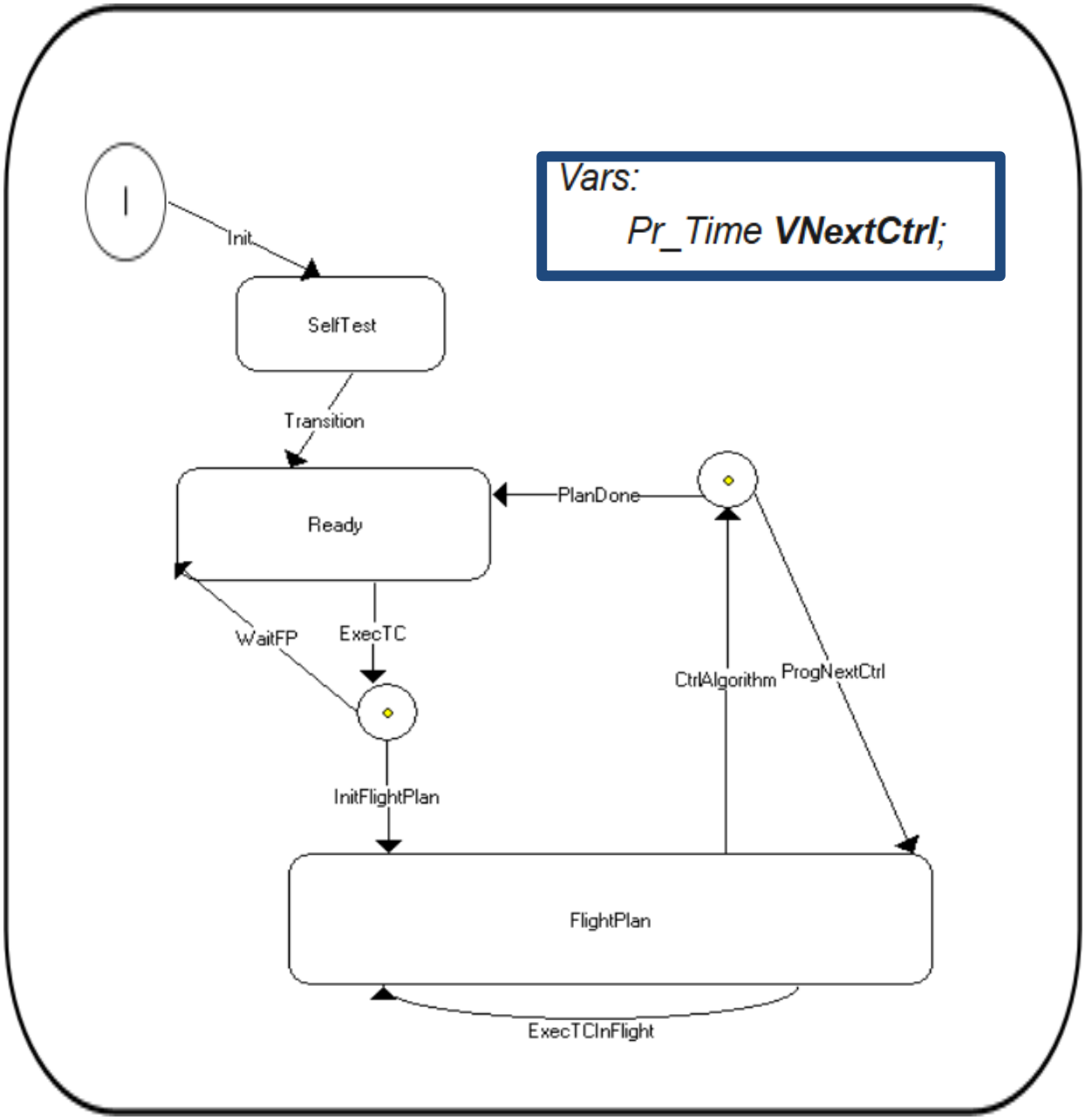


5.4.1.1 Behaviour

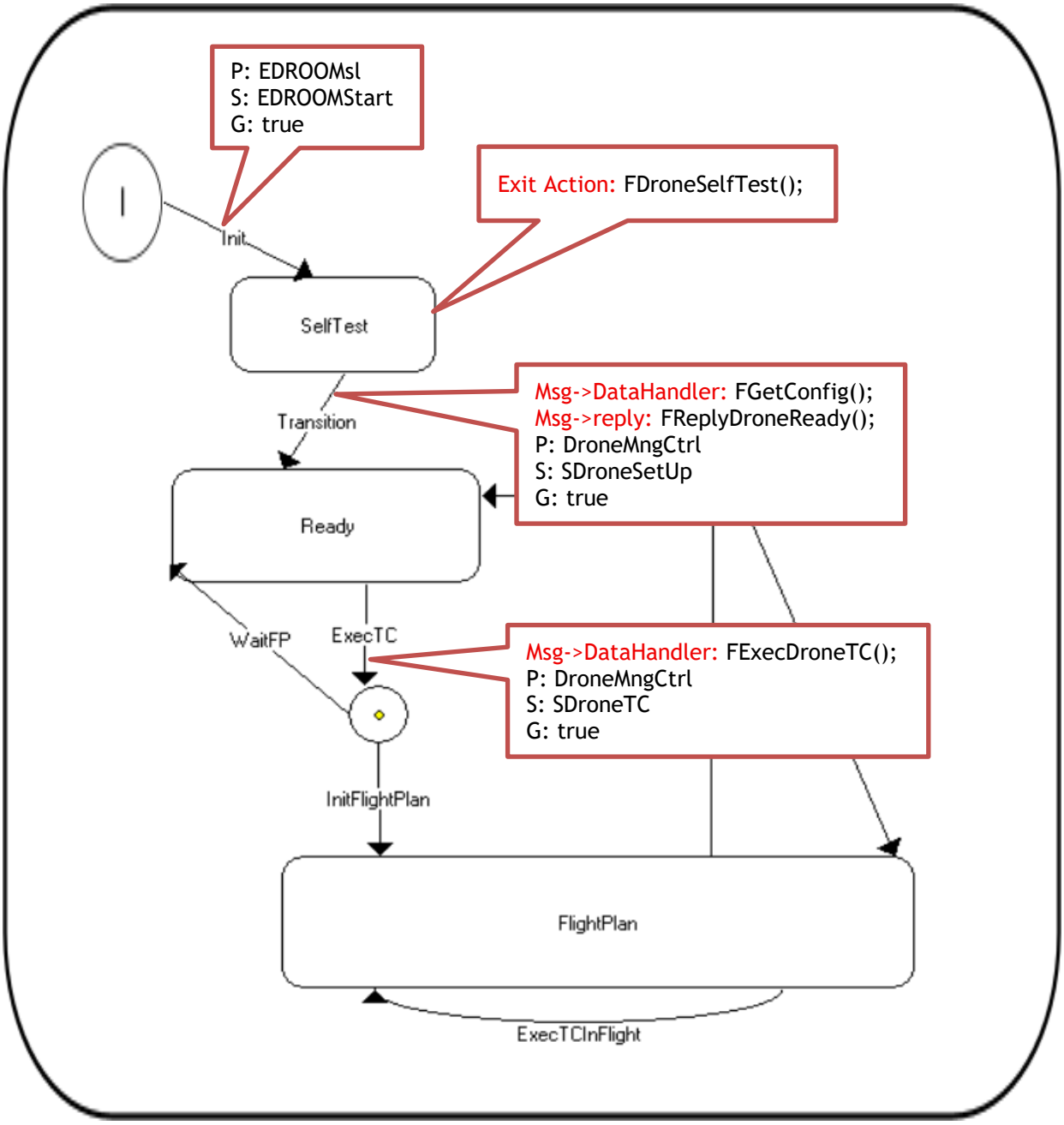
TODO 04 Añade el diagrama de estados de la clase CCDroneMng

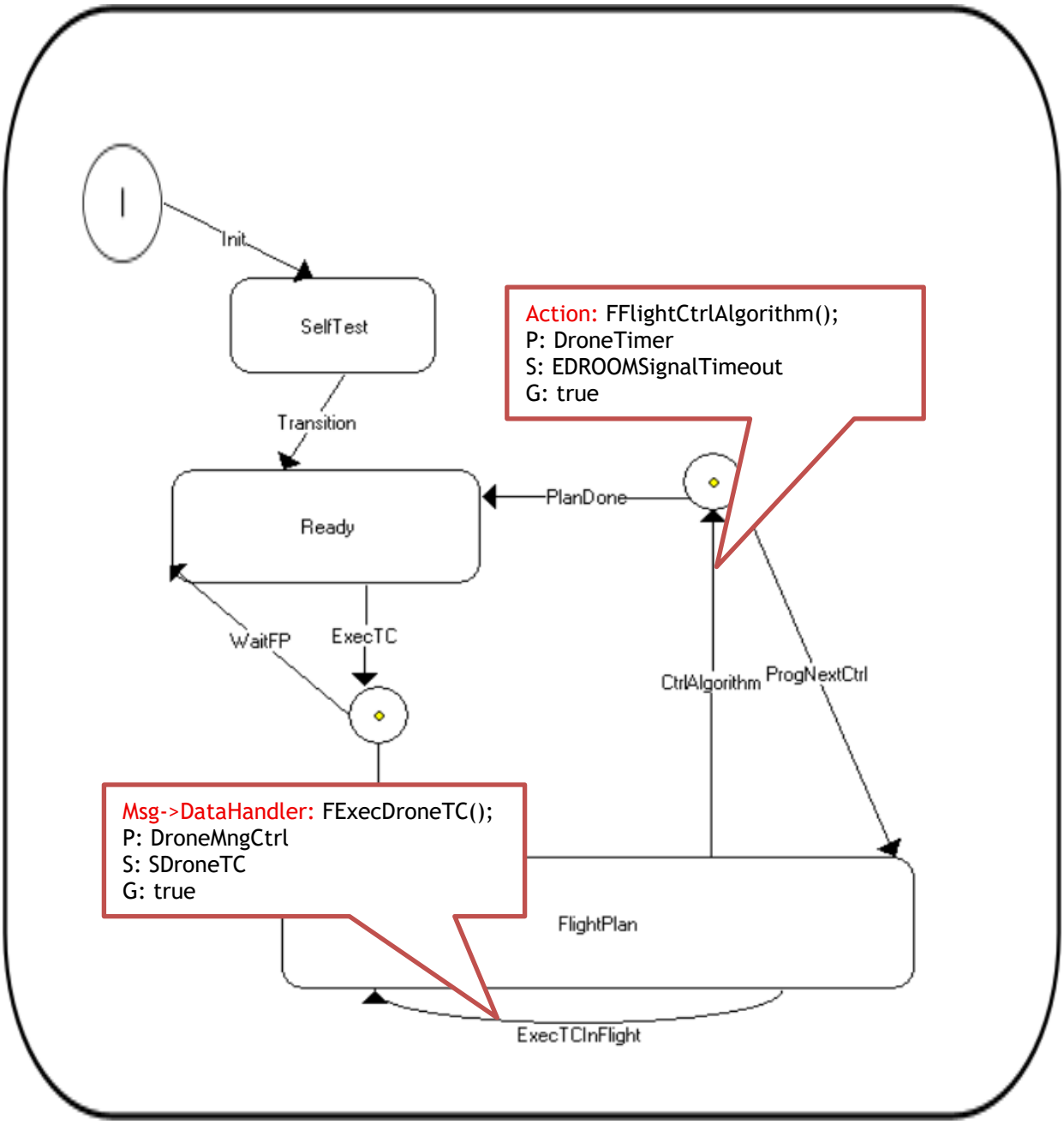


TODO 05 Añade las variables de la clase CCDroneMng

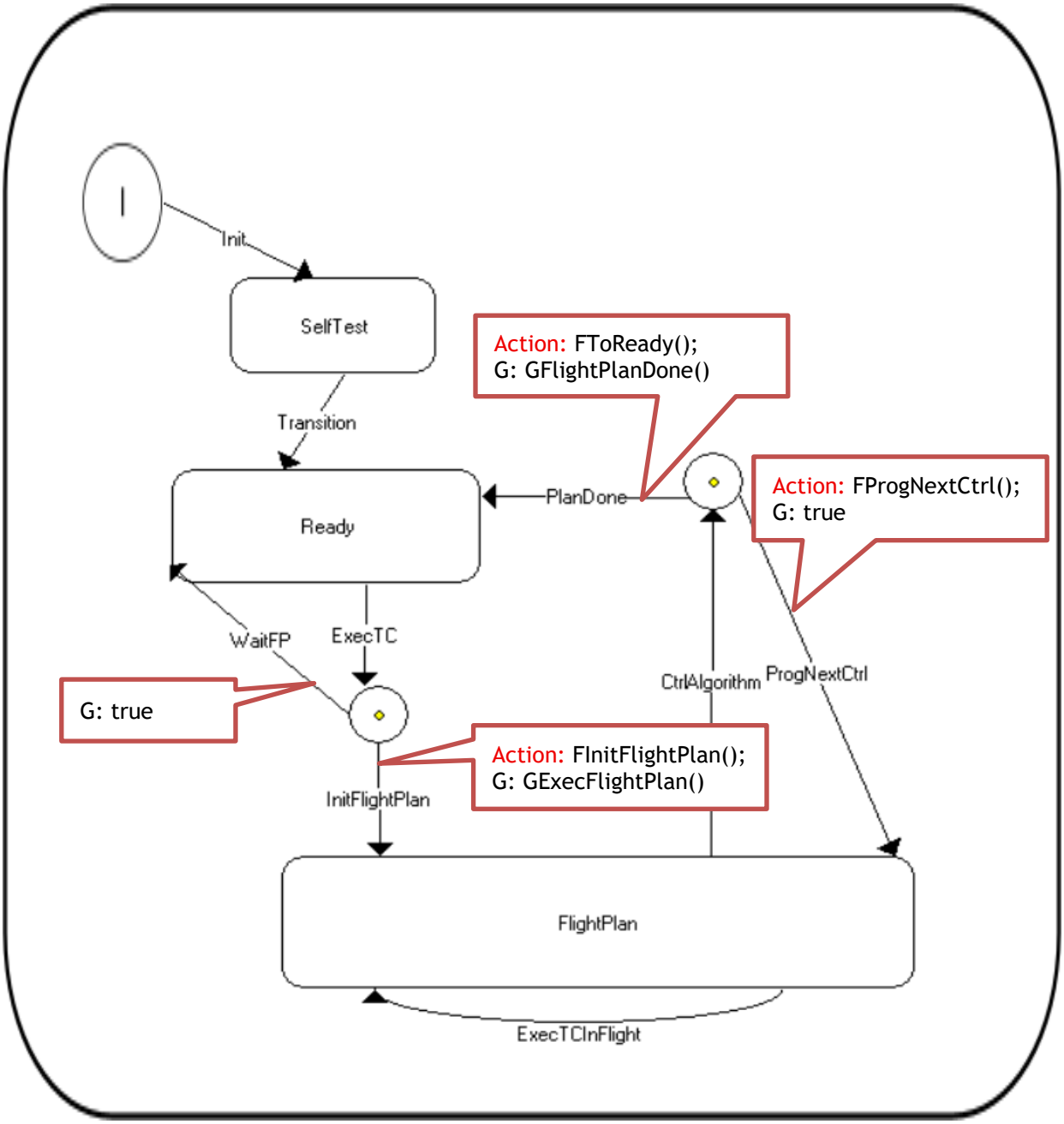


TODO 06 Añade la condición de disparo y la acción de cada una de las transiciones (utiliza el diagrama de estados varias veces si lo ves necesario)




Init
SelfTest
Transition
Ready
PlanDone
Action: FFlightCtrlAlgorithm();
P: DroneTimer
S: EDROOMSignalTimeout
G: true
WaitFP
ExecTC
CtrlAlgorithm
ProgNextCtrl
Msg->DataHandler: FExecDroneTC();
P: DroneMngCtrl
S: SDroneTC
G: true
FlightPlan
ExecTCInFlight

TODO 07 Añade la guarda y la acción asociada de cada una de las ramas de un choice points (utiliza el diagrama de estados varias veces si lo ves necesario)



TODO 08 Añade el código de las acciones y guardas (marca en azul el código que se genera automáticamente al solicitar un servicio)

```
void FDroneSelfTest() {

        pus_service129_drone_selftest();
}
```

```
void FGetConfig() {

        CDDroneConfig & varSDroneSetUp = *(CDDroneConfig *) Msg->data;

        pus_service129_setup(varSDroneSetUp);
}
```

```
void FReplyDroneReady() {

        pus_service129_drone_ready();

        Msg->Reply(SDroneReady);
}
```

```
void FExecDroneTC() {

        CDTCHandler & varSDroneTC = *(CDTCHandler *) Msg->data;

        varSDroneTC.ExecDroneTC();
}
```

```
void FFlightCtrlAlgorithm() {

        pus_service129_flight_ctrl_algorithm();
}
```

```
bool GExecFlightPlan() {

        return pus_service129_flight_plan_ready();
}
```

```
void FInitFlightPlan() {

        Pr_Time time;

        time.GetTime(); // Get current monotonic time
        time+=Pr_Time(0,100000); // Add X sec + Y microsec
        VNextCtrl=time;
        pus_service129_init_flight_plan();

        DroneTimer.InformAt(time);
}
```

```
bool GFlightPlanDone() {

        return pus_service129_flight_plan_done();
}
```

```
void FToReady() {

        pus_service129_drone_ready();
}
```

```
void FProgNextCtrl() {

        Pr_Time time;

        VNextCtrl+= Pr_Time(0,100000); // Add X sec + Y microsec
        time=VNextCtrl;

        DroneTimer.InformAt(time);
}
```